



DEVELOPERS LEVEL · GRADES 4-5

# Curriculum Guide

4 Semesters + Capstone · Complete Teaching Document

TEACHER EDITION

Scope & sequence · lesson plans · theory · projects · quizzes · alignment · inclusive design

# 1. Scope & Sequence

Developers is where students move from marking up pages to **making them think**. They review HTML/CSS and step into **JavaScript** — variables, functions, events, conditionals, and arrays — building five tools that solve real problems. Every semester weaves the five CODEship strands, with a deliberate lift in **math logic, financial literacy, media literacy, and AI/cyber awareness**. Students read independently at this age, so materials are text-and-code forward.

| Semester | Project                           | Coding focus                                           | Big idea                                |
|----------|-----------------------------------|--------------------------------------------------------|-----------------------------------------|
| 1        | Homework Timer & Focus Tool       | Variables, functions, events (onclick), console.log    | Code can store values and act on clicks |
| 2        | Healthy Snack Calculator          | Operators (+ - * /), input → output, variables         | Programs turn input into useful output  |
| 3        | Fact or Fake? Media-Literacy Quiz | if/else, booleans, arrays; algorithms & misinformation | Code decides — and so must we, online   |
| 4        | Budget Buddy                      | Arrays, loops, budget math (financial literacy)        | Lists + loops power real money math     |
| Capstone | Problem-Solver App                | Full JS toolkit + design thinking                      | Pick a real problem and build the tool  |

## How each semester runs

Each semester is **4 weekly classes** building to **1 project**, with a **theory unit, 4 worksheets**, a **project planner**, and **2 quizzes of 10 questions** (Quiz 1 formative; Quiz 2 summative) with a warm answer key. The **capstone** is the graduation gate to Engineers. Per level: 20 sessions · 5 projects · 8 quizzes · 4 theory units.

**S3 'Fact or Fake?' is the Exemplar semester** — the media-literacy and AI/algorithm content is built to full depth and sets the bar for the level.

## 2. Inclusive Design Playbook

Universal Design for Learning (UDL) is built into every lesson, and five accommodation profiles overlay on top. Both extend the platform's existing challenge-tags for reading, focus, and confidence.

**UDL baseline (all learners):** *Representation* — concepts shown as code, spoken, and diagrammed; run every example live. *Engagement* — real tools students actually want (timers, calculators, budgets), student choice of problem. *Expression* — show learning by coding, tracing, explaining, or pseudocoding — not only by writing prose.

**Accommodation profiles (Grade 4-5 specifics):** **ADHD/focus:** short code segments; run-and-see feedback; a checklist of the class's syntax; the Focus Tool itself models the skill. **ASD:** consistent editor routine; explicit success criteria; predictable file/function naming. **Dyslexia/dysgraphia:** syntax-highlighted starters; autocomplete; never penalize spelling in identifiers; speak-aloud tracing. **Anxiety/low-confidence:** pre-written starter code; "bugs are normal — that's debugging"; a guaranteed early success. **Gifted/2e:** a "Developer Extra" each class (add a feature, refactor a function, handle an edge case); no busywork.

These map to the platform's challenge-tags (reading-support, focus, confidence) plus tags for ASD-routine and gifted-extension. One core curriculum + a reusable accommodation layer delivers variances without multiplying documents.

## 3. Semester 1 — "Homework Timer & Focus Tool"

**Real-world hook:** a tool students actually want — a timer that helps them manage their own focus. **Coding focus:** JavaScript **variables**, **functions**, the `onclick` event, and `console.log` for debugging. **Big question:** *Can code store values and do something useful when I click?*

### Theory unit

A **variable** is a labelled box that stores a value; a **function** is named steps you run again; an **event handler** (`onclick`) runs a function on a click; `console.log` prints values to debug. This is the foundation of interactivity.

### Lesson plans

**Class 1 — Variables.** Obj: declare and read variables. Align: Math C3 variables; Language. Warm-up: 'what would a timer need to remember?'. Teach: `let minutes = 25;`. Coding space: change a variable and re-run. Practice: match/ write variables. Reflect: why name it clearly? Profiles: dyslexia — starter with names. Assess: write a seconds variable.

**Class 2 — Functions.** Obj: define and call a function. Align: C3 defined processes. Warm-up: name the steps of 'start the timer'. Teach: `function start(){}.` Coding space: wrap steps in a function and call it. Practice: order + name functions. Reflect: why reuse steps? Profiles: ADHD — one function at a time. Extra: add `reset()`.

**Class 3 — Events.** Obj: run a function on click. Align: C3 events. Warm-up: 'what should Start do?'. Teach: `onclick`. Coding space: wire a button to `start()`. Practice: match buttons to functions. Reflect: what triggers your code? Profiles: anxiety — starter button provided. Assess: identify the `onclick` line.

**Class 4 — Plan my timer.** Obj: design the tool. Align: SEL self-management; Design. Warm-up: sketch the screen. Teach: from plan to build. Coding space: assemble variables + functions + button. Practice: planning sheet. Reflect: does it help me focus? Profiles: all — present to a partner. Assess: exit — name one variable and one function.

## Project outline

Students build a focus timer: time stored in variables, functions run by button clicks, and a message when time is up. Success criteria: a working variable, a called function, a click event, and a spoken explanation of how it aids focus. Extension: add pause/reset or a session counter.

## Rubric (score 1-4 each)

Uses a variable

Uses a function

Responds to a click

Explains how it helps focus

## Ontario alignment

Math C3 Coding (Gr 4-5): variables, defined processes, events

Language: reading/oral explanation of code

SEL (Strand A): self-management and focus

## 4. Semester 2 — "Healthy Snack Calculator"

**Real-world hook:** an input-driven tool for real nutrition choices — the input → calculate → output pattern. **Coding focus: Operators** (+ - \* /), **input** and **output**, and variables that hold each number. **Big question:** *Can a program take what I type and give me a useful answer?*

### Theory unit

Tools follow a pattern: take **input**, do a **calculation** with **operators**, show **output**. Variables hold the numbers. This joins arithmetic fluency with real health literacy.

### Lesson plans

**Class 1 — Operators.** Obj: use + - \* /. Align: Math B number/operations. Warm-up: mental math race. Teach: operators in code. Coding space: compute a total. Practice: match + solve. Reflect: which operator for 'per serving × servings'? Profiles: dyscalculia-aware — calculator allowed. Assess: solve  $90 \times 3$ .

**Class 2 — Input & output.** Obj: distinguish input from output. Align: C3 input/output. Warm-up: label a calculator's parts. Teach: read a value in, show a value out. Coding space: read input, print output. Practice: label worksheet. Reflect: what's the user's input? Profiles: ASD — same template. Assess: mark input vs output.

**Class 3 — Build a formula.** Obj: translate a real question to a formula. Align: Math patterning/algebra. Warm-up: word problem → expression. Teach: `total = per * servings`. Coding space: type the formula. Practice: fill formulas. Reflect: is the math right? Profiles: anxiety — worked example. Extra: add a % daily-goal calc.

**Class 4 — Plan my calculator.** Obj: design inputs, formula, output. Align: Health literacy; Design. Warm-up: pick a real snack question. Teach: plan first. Coding space: assemble the tool. Practice: planning sheet. Reflect: does the output help a choice? Profiles: all — partner test. Assess: state your formula.

### Project outline

Students build a calculator that takes input, computes with operators, and outputs a helpful nutrition figure. Success criteria: real input, correct operators, clear output, and a real choice it supports. Extension: add a second calculation or a healthy-range message.

## Rubric (score 1-4 each)

Takes input

Uses operators correctly

Shows clear output

Supports a real choice

## Ontario alignment

Math C3 Coding (Gr 4-5): input/output, operators

Math B: operations fluency

Health & Physical Education: healthy eating literacy

## 5. Semester 3 — "Fact or Fake? Media-Literacy Quiz"

**Real-world hook:** a game that trains players to spot misinformation — the level's Exemplar semester. **Coding focus:** **if/else**, **booleans**, **arrays**; plus how **algorithms** amplify content and how to check a source. **Big question:** *Can code decide — and can we decide what to trust online?*

### Theory unit

**if/else** chooses a path based on a **boolean** test; a quiz stores questions in an **array** and checks each answer. The AI/cyber literacy core: platform **algorithms** amplify engaging content, and **misinformation** (including AI-generated media) can look real — verify the source before trusting or sharing.

### Lesson plans

**Class 1 — If/else.** Obj: branch on a condition. Align: C3 conditionals. Warm-up: 'if it rains, then...'. Teach: if/else + ==. Coding space: score a correct answer. Practice: complete conditions. Reflect: when does else run? Profiles: dyslexia — highlighted syntax. Assess: write an equality test.

**Class 2 — Booleans.** Obj: use true/false. Align: C3 boolean logic; Math logic. Warm-up: true/false statements. Teach: boolean variables. Coding space: set correct = true. Practice: mark statements. Reflect: how do booleans drive if? Profiles: ASD — explicit examples. Assess: write a boolean.

**Class 3 — Spot the fake.** Obj: evaluate sources. Align: Language media literacy; Cyber. Warm-up: real vs fake headline. Teach: source-checking (stop, investigate the source). Coding space: add a 'why check?' tip to the quiz. Practice: sort real/needs-checking. Reflect: what made you doubt it? Profiles: anxiety — no wrong takes, discuss. Assess: name two ways to verify.

**Class 4 — Plan my quiz.** Obj: design the quiz. Align: C3 arrays; Language. Warm-up: pick a topic. Teach: questions array + if/else feedback. Coding space: assemble it. Practice: planning sheet. Reflect: does it teach checking? Profiles: gifted — add a difficulty level. Assess: outline your array.

### Project outline

Students build a quiz that stores questions in an array, checks answers with if/else, and teaches one way to verify a source. Success criteria: working if/else, answer



checking, a media-literacy tip, and an explanation of one algorithm/AI idea.  
Extension: track score with a variable and show a final message.

### Rubric (score 1-4 each)

Uses if/else

Checks answers (array/boolean)

Teaches spotting fakes

Explains an algorithm/AI idea

### Ontario alignment

Math C3 Coding (Gr 4-5): conditionals, arrays, boolean logic

Language: media literacy; critical thinking

Technology & Society: algorithms, misinformation, AI-generated media

## 6. Semester 4 — "Budget Buddy"

**Real-world hook:** a savings and spending tracker — real financial literacy through arrays and loops. **Coding focus:** **Arrays, loops** to total them, and budget math (income – expenses = balance). **Big question:** *Can lists and loops help me manage real money?*

### Theory unit

An **array** holds many values (each expense); a **loop** repeats a step for every item to total them. Financial literacy: **income – expenses = balance**, and telling **needs** from **wants** (Ontario Math Strand F).

### Lesson plans

**Class 1 — Arrays.** Obj: build and read an array. Align: C3 arrays/lists. Warm-up: list this week's spends. Teach: [5,3,8] and indexing. Coding space: make an expenses array. Practice: read items. Reflect: why a list not many variables? Profiles: dyslexia — sample array. Assess: give the first item.

**Class 2 — Loops.** Obj: total with a loop. Align: C3 loops/repetition. Warm-up: add a list by hand. Teach: for loop sums. Coding space: total the array. Practice: trace the running total. Reflect: why loops scale? Profiles: ADHD — step-through. Extra: find the max expense.

**Class 3 — Budget math.** Obj: compute balance; needs vs wants. Align: Math Strand F financial literacy. Warm-up: needs or wants? Teach: income – expenses. Coding space: compute balance. Practice: sort + compute. Reflect: what does a positive balance let you do? Profiles: anxiety — worked example. Assess: compute a balance.

**Class 4 — Plan Budget Buddy.** Obj: design the tracker. Align: Strand F; Design. Warm-up: pick a saving goal. Teach: plan the data + math + message. Coding space: assemble it. Practice: planning sheet. Reflect: is the advice helpful? Profiles: all — partner test. Assess: state your saving tip.

### Project outline

Students build a tracker: expenses in an array, totalled by a loop, with a balance and a saving message. Success criteria: an array, a loop total, correct budget math, and a saving choice explained. Extension: separate needs/wants or add a goal tracker.

## Rubric (score 1-4 each)

Uses an array

Totals with a loop

Correct budget math

Explains a saving choice

## Ontario alignment

Math C3 Coding (Gr 4-5): arrays, loops

Math Strand F: financial literacy (budgeting, saving)

Language: explaining reasoning

## 7. Capstone — "Problem-Solver App"

**Real-world hook:** students pick a real problem in their own life and build a JavaScript tool for it — the graduation gate. **Coding focus:** The full JS toolkit (variables, functions, if/else, arrays, events) applied through **design thinking**. **Big question:** *Can I identify a real problem and build a working tool that helps?*

### Theory unit

**Design thinking** — understand the problem and **user**, plan, build a **prototype**, test and **debug**, improve. Students integrate everything from the level to ship a small but real tool.

### Lesson plans

**Class 1 — Find a problem.** Obj: choose a real, scoped problem. Align: Design; Language. Warm-up: list daily frictions. Teach: problem + user. Coding space: create the project. Practice: problem statement. Reflect: who is it for? Profiles: anxiety — idea menu. Assess: name your user.

**Class 2 — Plan the solution.** Obj: choose tools + sketch. Align: C3 planning. Warm-up: which tools fit? Teach: map features to tools. Coding space: scaffold files. Practice: plan sheet. Reflect: simplest version first. Profiles: ASD — template. Assess: list your tools.

**Class 3 — Build & trace.** Obj: pseudocode then build. Align: C3 defined processes; debugging. Warm-up: write the main steps. Teach: pseudocode → code. Coding space: build the core function. Practice: trace it. Reflect: what to console.log? Profiles: gifted — add an edge case. Assess: show your main().

**Class 4 — Test & present.** Obj: test, debug, pitch. Align: C3 debugging; Language presentation. Warm-up: peer test. Teach: test checklist + pitch. Coding space: fix a bug. Practice: graduation checklist. Reflect: what did you fix? Profiles: all — present to a partner first. Assess: 30-second pitch.

### Project outline

Graduation gate: a working JavaScript tool for a real, student-chosen problem, using three or more tools, tested and presented. Passing (all four rubric criteria) graduates the student to Engineers.

## Rubric (score 1-4 each)

Solves a real problem

Uses 3+ JS tools

Works (tested/debugged)

Presents & explains

## Ontario alignment

Math C3 Coding (Gr 4-5): integrate variables, events, conditionals, arrays; debugging

Language: planning and oral presentation

Design: design-thinking process

## 8. Alignment & Compliance

Developers is built on the pan-Canadian five-area spine and tagged first to **Ontario**. Coding maps to **Mathematics Strand C3 (Coding), Grades 4-5** — variables, defined processes, events, conditional statements, and repeating events (loops). **Financial literacy** maps to **Strand F** (Budget Buddy). Literacy maps to the **Language** curriculum (media literacy, critical thinking, explanation). AI and cyber literacy live in **Technology & Society** (algorithms, misinformation, AI-generated media). Deeper Python, hands-on AI, and real cybersecurity arrive in Engineers.

**Compliance language (locked):** always describe materials as "**aligned to / maps to / supports**" provincial expectations — **never** "endorsed by" or "approved by" a ministry. Validate each alignment cell against the current official document before any public claim.

### Cross-cutting (level-wide)

UDL is baked into every lesson; the five accommodation profiles overlay as reusable one-pagers. Provincial crosswalks (BC ADST, Alberta, Quebec + French) map over this same core content. Lessons, projects, quizzes, and alignment tags export to the platform schema with the coding space set to JavaScript (with HTML/CSS review) for this level.

DREAM. CODE. ACHIEVE. — Developers turns markup into logic: from a first variable to a real problem-solving app. Next stop: Engineers (Python, AI, and cybersecurity).